

# **PERANCANGAN DAN IMPLEMENTASI WEB SERVER AMAN BERBASIS CONTAINER MENGGUNAKAN DOCKER, NGINX, DAN MARIADB**

*Dosen Pengampu : Ainul Hizriadi, S.Kom., M.Sc*



Disusun Oleh

KOM C – KELOMPOK 2

Anggota :

|                           |             |
|---------------------------|-------------|
| Dolly Efredi Bukit        | (241402021) |
| Rohaya Hasibuan           | (241402030) |
| Cindy Samosir             | (241402009) |
| Chaterine Eklesia Maryati | (241402123) |
| Earlin Tasia Gulo         | (241402089) |
| Cindy Artika              | (241402012) |

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS ILMU KOMPUTER DAN TEKNOLOGI  
INFORMASI  
UNIVERSITAS SUMATERA UTARA  
2026**

## DAFTAR ISI

|                                                     |           |
|-----------------------------------------------------|-----------|
| DAFTAR ISI .....                                    | ii        |
| <b>BAB I PENDAHULUAN.....</b>                       | <b>1</b>  |
| 1.1 Latar Belakang .....                            | 1         |
| 1.2 Tujuan .....                                    | 1         |
| <b>BAB II DESAIN SISTEM .....</b>                   | <b>2</b>  |
| 2.1 Arsitektur Sistem.....                          | 2         |
| 2.2 Diagram Jaringan dan Container.....             | 3         |
| 2.3 Struktur Folder Proyek.....                     | 4         |
| 2.4 Desain Keamanan Sistem.....                     | 4         |
| <b>BAB III IMPLEMENTASI DAN KONFIGURASI .....</b>   | <b>5</b>  |
| 3.1 Install Docker .....                            | 6         |
| 3.2 Struktur Folder .....                           | 7         |
| 3.3 File Environment .env .....                     | 7         |
| 3.4 Generate SSL .....                              | 7         |
| 3.5 Konfigurasi Nginx Reverse Proxy.....            | 8         |
| 3.6 Docker Compose .....                            | 7         |
| 3.7 Instalasi WordPress.....                        | 10        |
| 3.8 DNS Server (Bind9).....                         | 11        |
| 3.9 UFW Firewall .....                              | 11        |
| 3.10 SSH Hardening .....                            | 12        |
| 3.11 Fail2Ban .....                                 | 12        |
| <b>BAB IV HASIL IMPLEMENTASI .....</b>              | <b>13</b> |
| 4.1 UFW Aktif dan Dikonfigurasi (Default Deny)..... | 13        |
| 4.2 SSL/TLS Valid (HTTPS).....                      | 14        |
| 4.3 Fail2Ban Aktif.....                             | 15        |
| 4.4 OS Hardening (sysctl).....                      | 16        |
| 4.5 Database Tidak Diekspos ke Luar Host.....       | 16        |
| 4.6 Minimal 3 Kontainer Terpisah.....               | 17        |
| 4.7 Network Bridge Isolation .....                  | 18        |
| 4.8 Kontainer Berjalan Non-Root.....                | 18        |
| 4.9 File docker-compose.yml Valid.....              | 19        |

**BAB V KENDALA DAN SOLUSI ..... 20**

5.1 Kendala yang Dihadapi.....20

5.2 Solusi yang Diterapkan.....20

**BAB VI PEMBAGIAN TUGAS ..... 21**

**BAB VII PENUTUP ..... 22**

**LAMPIRAN ..... 23**

- Link GitHub
- Link YouTube

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Perkembangan teknologi server dan layanan web saat ini menuntut adanya sistem yang tidak hanya berjalan dengan baik, tetapi juga memiliki tingkat keamanan yang memadai. Keamanan server dan jaringan menjadi aspek penting karena layanan web rentan terhadap berbagai ancaman seperti akses ilegal, serangan brute force, maupun kebocoran data. Oleh karena itu, diperlukan penerapan sistem keamanan yang mampu melindungi layanan dan data yang terdapat di dalam server.

Pada tugas besar ini, kelompok kami membangun layanan web berbasis WordPress menggunakan teknologi containerisasi Docker dengan arsitektur tiga container yang terdiri dari Nginx sebagai reverse proxy, WordPress sebagai aplikasi web, dan MariaDB sebagai database server. Selain itu, sistem juga dilengkapi dengan penerapan SSL/TLS untuk HTTPS, firewall UFW, SSH hardening, serta Fail2Ban untuk meningkatkan keamanan server dan jaringan.

Dengan adanya implementasi tersebut, diharapkan sistem yang dibangun dapat berjalan secara stabil, terisolasi, dan lebih aman sesuai dengan konsep keamanan server modern serta memenuhi kebutuhan pada mata kuliah Keamanan Server dan Jaringan.

### **1.2 Tujuan**

Tujuan dari pelaksanaan tugas besar ini adalah:

1. Membangun layanan web berbasis WordPress menggunakan Docker container.
2. Mengimplementasikan arsitektur tiga container yang terdiri dari Nginx, WordPress, dan MariaDB.
3. Menerapkan keamanan server menggunakan SSL/TLS, UFW Firewall, SSH Hardening, dan Fail2Ban.
4. Mengisolasi layanan menggunakan Docker network agar lebih aman.
5. Memahami proses implementasi keamanan server dan jaringan pada lingkungan Linux.

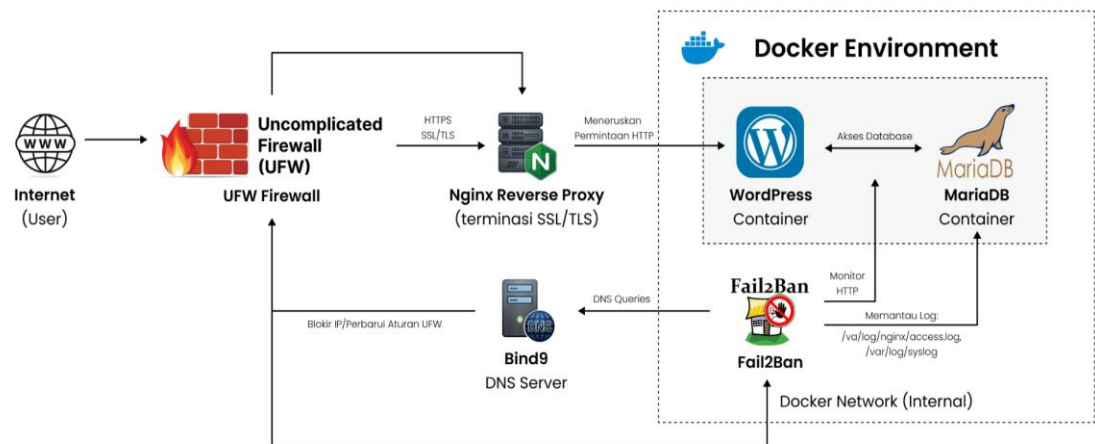
## BAB II

### DESAIN SISTEM

#### 2.1 Arsitektur Sistem

Pada tugas besar ini, sistem dibangun menggunakan arsitektur berbasis container dengan Docker. Sistem terdiri dari tiga container utama, yaitu Nginx sebagai reverse proxy, WordPress sebagai aplikasi web, dan MariaDB sebagai database server. Seluruh layanan dijalankan menggunakan Docker Compose agar proses pengelolaan container menjadi lebih mudah dan terstruktur.

Berikut merupakan diagram arsitektur sistem yang digunakan pada proyek ini:



**Gambar 2.1** Desain arsitektur sistem beserta diagram jaringan dan kontainer

Pada arsitektur tersebut, client mengakses website melalui protokol HTTPS menuju container Nginx. Nginx berfungsi sebagai reverse proxy yang menerima request dari client kemudian meneruskannya ke container WordPress. Selanjutnya, WordPress akan terhubung dengan MariaDB untuk proses penyimpanan dan pengelolaan data website.

## 2.2 Diagram Jaringan dan Container

Sistem menggunakan dua Docker network, yaitu frontend dan backend. Network frontend digunakan untuk komunikasi antara client dan Nginx, sedangkan network backend digunakan untuk komunikasi internal antara Nginx, WordPress, dan MariaDB.

Container MariaDB hanya terhubung pada network backend dan tidak membuka port ke luar host. Hal ini dilakukan untuk meningkatkan keamanan database dan mencegah akses langsung dari luar server.

Dengan penggunaan Docker bridge network yang terpisah, setiap layanan dapat berjalan lebih terisolasi sehingga keamanan sistem menjadi lebih baik.

Struktur direktori proyek yang digunakan dalam implementasi sistem adalah sebagai berikut:

Kel-5/

|

|— nginx/

| |— conf.d/

| |— ssl/

|

|— wordpress/

|

|— mariadb/

|

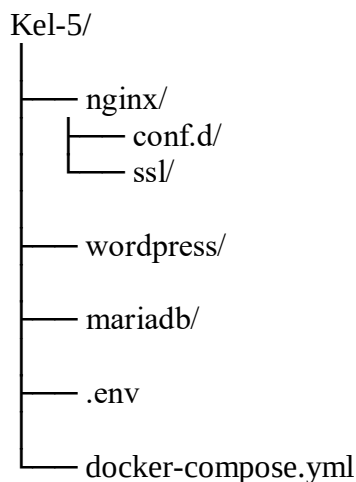
|— .env



Folder `nginx/conf.d` digunakan untuk menyimpan konfigurasi reverse proxy Nginx, sedangkan folder `nginx/ssl` digunakan untuk menyimpan sertifikat SSL/TLS. File `.env` digunakan untuk menyimpan konfigurasi database seperti username dan password agar lebih aman dan terpisah dari konfigurasi utama.

### 2.3 Struktur Folder Proyek

Struktur direktori proyek yang digunakan dalam implementasi sistem adalah sebagai berikut:



Folder `nginx/conf.d` digunakan untuk menyimpan konfigurasi reverse proxy Nginx, sedangkan folder `nginx/ssl` digunakan untuk menyimpan sertifikat SSL/TLS. File `.env` digunakan untuk menyimpan konfigurasi database seperti username dan password agar lebih aman dan terpisah dari konfigurasi utama.

### 2.4 Desain Keamanan Sistem

Pada sistem ini diterapkan beberapa mekanisme keamanan untuk melindungi layanan server dan jaringan. HTTPS diterapkan menggunakan SSL/TLS agar komunikasi data

terenkripsi. Firewall UFW digunakan untuk membatasi akses port yang diperbolehkan pada server.

Selain itu, SSH Hardening diterapkan menggunakan autentikasi key pair untuk meningkatkan keamanan akses SSH. Sistem juga menggunakan Fail2Ban untuk mencegah serangan brute force pada layanan SSH. Database MariaDB tidak diekspos ke luar host dan komunikasi antar layanan dipisahkan menggunakan Docker network untuk meningkatkan keamanan sistem secara keseluruhan.

## BAB III

### IMPLEMENTASI DAN KONFIGURASI

#### 3.1 Install Docker

```
sudo apt update  
sudo apt upgrade -y
```

Install Docker : `curl -fsSL https://get.docker.com | sudo sh`  
Tambahkan user : `sudo usermod -aG docker $USER`

Logout lalu login kembali, Verifikasi:  
`docker --version`  
`docker compose version`

#### 3.2 Struktur Folder

Buat direktori proyek:  
`mkdir ~/wordpress-secure`  
`cd ~/wordpress-secure`

Struktur :  
Kel-5/

```
|  
|— nginx/  
|   |— conf.d/  
|   |— ssl/  
|  
|— wordpress/  
|  
|— mariadb/  
|  
|— .env  
|  
|— docker-compose.yml
```

### 3.3 File Environment

Buat : nano .env

Isi:

MYSQL\_ROOT\_PASSWORD=12345

MYSQL\_DATABASE=wordpressdb

MYSQL\_USER=wpuser

MYSQL\_PASSWORD=12345

### 3.4 Generate SSL

Buat folder : mkdir -p nginx/ssl

Generate sertifikat :

openssl req -x509 \

-nodes \

-days 365 \

-newkey rsa:2048 \

-keyout nginx/ssl/selfsigned.key \

-out nginx/ssl/selfsigned.crt

### 3.5 Konfigurasi Nginx Reverse Proxy

Buat folder : mkdir -p nginx/conf.d

File: nano nginx/conf.d/default.conf

Isi:

server {

listen 80;

server\_name \_;

return 301 https://\$host\$request\_uri;

}

server {

```

listen 443 ssl;
server_name _;

ssl_certificate /etc/nginx/ssl/selfsigned.crt;
ssl_certificate_key /etc/nginx/ssl/selfsigned.key;
ssl_protocols TLSv1.2 TLSv1.3;

location / {
    proxy_pass http://wordpress:80;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto https;
    proxy_set_header X-Real-IP $remote_addr;
}
}

```

Konfigurasi ini untuk :

- redirect HTTP → HTTPS
- SSL/TLS aktif
- reverse proxy ke WordPress

### 3.6 Docker Compose

Buat : nano docker-compose.yml

Isi:

services:

nginx:

```

image: nginx:latest
container_name: nginx-proxy
restart: always

```

ports:

- "80:80"
- "443:443"

volumes:

- ./nginx/conf.d:/etc/nginx/conf.d

- ./nginx/ssl:/etc/nginx/ssl

depends\_on:

- wordpress

networks:

- frontend
- backend

wordpress:

image: wordpress:latest

container\_name: wordpress

restart: always

environment:

WORDPRESS\_DB\_HOST: mariadb:3306

WORDPRESS\_DB\_USER: \${MYSQL\_USER}

WORDPRESS\_DB\_PASSWORD: \${MYSQL\_PASSWORD}

WORDPRESS\_DB\_NAME: \${MYSQL\_DATABASE}

volumes:

- wordpress\_data:/var/www/html

networks:

- backend

mariadb:

image: mariadb:11

container\_name: mariadb

restart: always

env\_file:

- .env

volumes:

- mariadb\_data:/var/lib/mysql

expose:  
- "3306"

networks:  
- backend

volumes:

wordpress\_data:

mariadb\_data:

networks:

frontend:

driver: bridge

backend:

driver: bridge

NOTES : ports: - "3306:3306"

Artinya database tidak bisa diakses dari luar host dan memenuhi syarat keamanan pada tugas.

### 3.7 Instalasi WordPress

Buka browser: [https://IP\\_VM](https://IP_VM)

Contoh: <https://172.30.67.107>

Karena sertifikat self-signed:

- Advanced
- Proceed

Lanjutkan wizard WordPress:

- Site Title
- Username Admin

- Password
- Email

Tekan tombol Install.

### 3.8 DNS Server (Bind9)

Install : `sudo apt install bind9 bind9-utils -y`

Edit : `sudo nano /etc/bind/named.conf.local`

Tambahkan:

```
zone "kelima.com" {
```

```
    type master;
```

```
    file "/etc/bind/db.kelima.com";
};
```

Copy template:

`sudo cp /etc/bind/db.local /etc/bind/db.kelima.com`

Edit: `sudo nano /etc/bind/db.kelima.com`

Isi: \$TTL 604800

```
@ IN SOA ns.kelima.com. root.kelima.com. (
2
```

```
604800
```

```
86400
```

```
2419200
```

```
604800
```

```
)
```

```
@ IN NS ns.kelima.com.
```

```
@ IN A 172.30.67.107
```

```
ns IN A 172.30.67.107
```

```
www IN A 172.30.67.107
```

Restart : `sudo systemctl restart bind9`

Test : `dig kelima.com`

### 3.9 UFW Firewall

Default policy:

`sudo ufw default deny incoming`

`sudo ufw default allow outgoing`

Allow service penting:

`sudo ufw allow 22/tcp`

`sudo ufw allow 80/tcp`

`sudo ufw allow 443/tcp`

`sudo ufw allow 53`

Aktifkan : `sudo ufw enable`

Verifikasi : `sudo ufw status verbose`

### **3.10 SSH Hardening**

Buat key pair dari komputer host : `ssh-keygen`

Salin public key ke VM:

`mkdir -p ~/.ssh`

`nano ~/.ssh/authorized_keys`

Setelah itu Paste key.

Permission :

`chmod 700 ~/.ssh`

`chmod 600 ~/.ssh/authorized_keys`

Restart : `sudo systemctl restart ssh`

### **3.11 Fail2Ban**

Buat : `sudo nano /etc/fail2ban/jail.local`

Isi:

[sshd]

`enabled = true`

`port = 22`

`maxretry = 3`

`findtime = 10m`

`bantime = 1h`

Restart : `sudo systemctl restart fail2ban`

Cek : `sudo fail2ban-client status`

## BAB IV

### HASIL IMPLEMENTASI

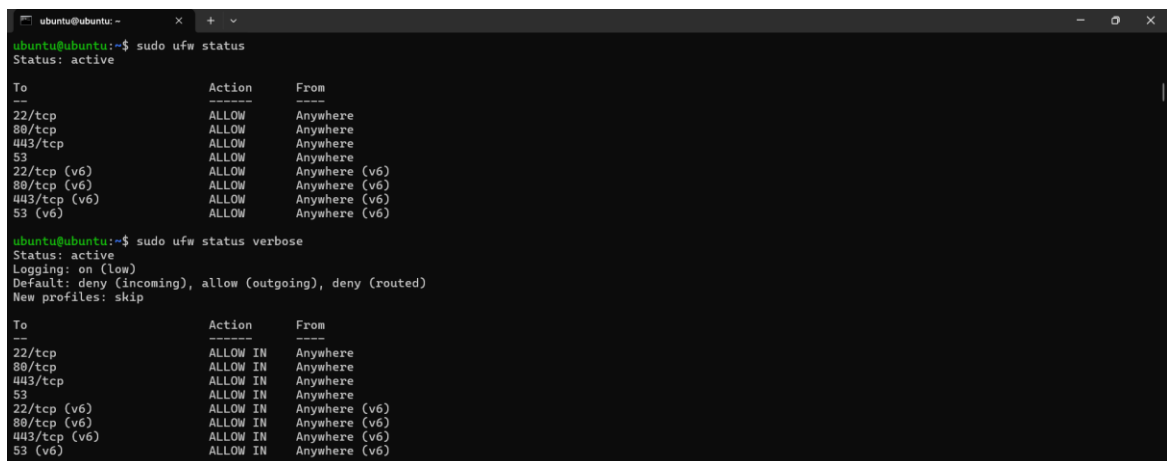
#### 4.1 UFW Aktif dan Dikonfigurasi (Default Deny)

Firewall UFW berhasil diaktifkan dan dikonfigurasi dengan kebijakan default berupa deny incoming dan allow outgoing. Konfigurasi ini bertujuan untuk membatasi akses masuk ke server sehingga hanya port tertentu yang diperbolehkan untuk diakses.

Port yang diizinkan pada sistem meliputi:

- Port 22 untuk SSH
- Port 80 untuk HTTP
- Port 443 untuk HTTPS
- Port 53 untuk DNS

Verifikasi dilakukan menggunakan perintah `sudo ufw status` dan `sudo ufw status verbose`. Hasil menunjukkan bahwa firewall aktif dan berjalan sesuai konfigurasi keamanan yang diterapkan.



```
ubuntu@ubuntu:~$ sudo ufw status
Status: active

To Action From
--
22/tcp ALLOW Anywhere
80/tcp ALLOW Anywhere
443/tcp ALLOW Anywhere
53 ALLOW Anywhere
22/tcp (v6) ALLOW Anywhere (v6)
80/tcp (v6) ALLOW Anywhere (v6)
443/tcp (v6) ALLOW Anywhere (v6)
53 (v6) ALLOW Anywhere (v6)

ubuntu@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
80/tcp ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
53 ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)
80/tcp (v6) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
53 (v6) ALLOW IN Anywhere (v6)
```

**Gambar 4.1** UFW aktif & dikonfigurasi (default deny)

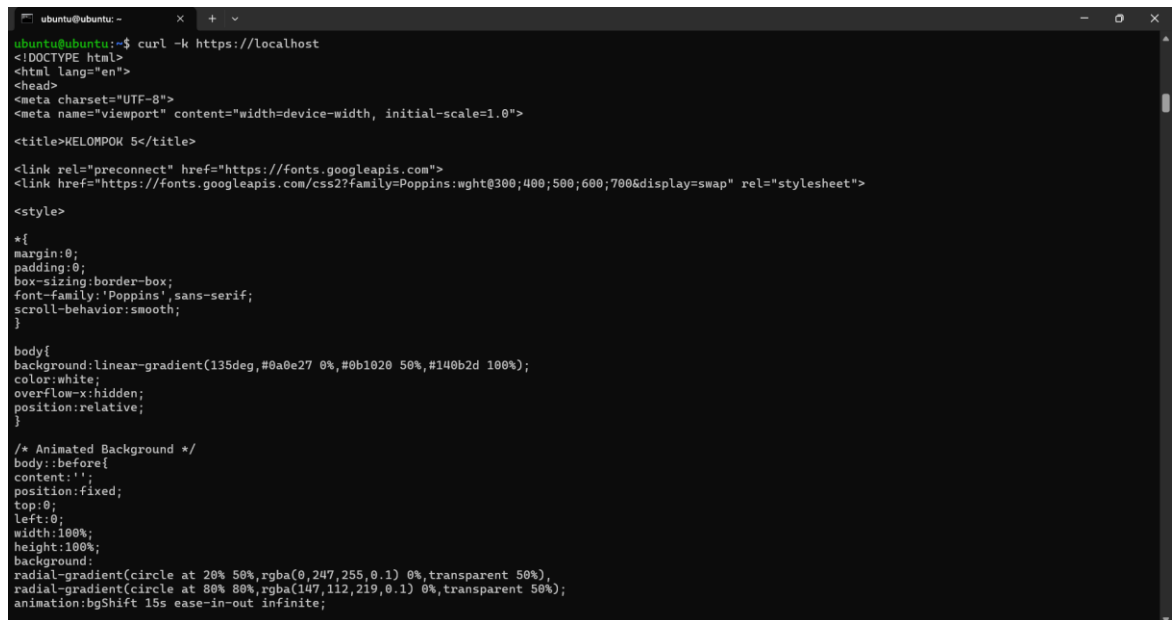
## 4.2 SSL/TLS Valid (HTTPS)

Sistem berhasil menerapkan SSL/TLS menggunakan sertifikat self-signed pada Nginx reverse proxy sehingga website dapat diakses menggunakan protokol HTTPS.

Pengujian dilakukan menggunakan perintah:

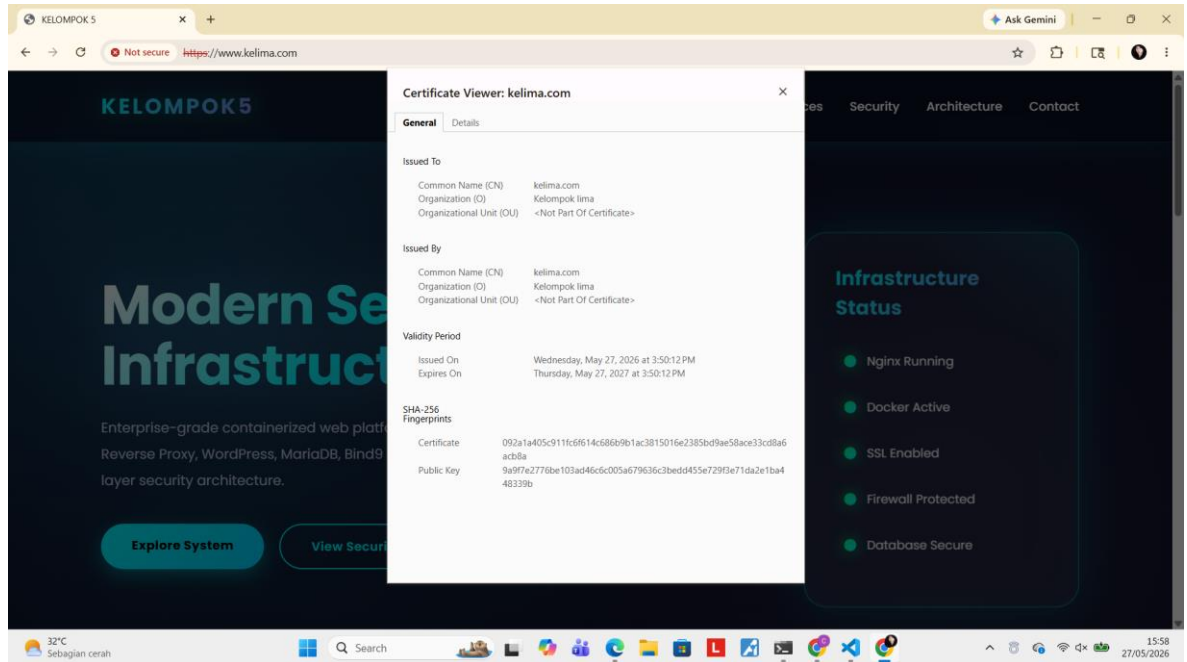
```
curl -k https://localhost
```

Hasil pengujian menampilkan source code website yang menunjukkan bahwa layanan HTTPS berhasil berjalan dengan baik. Selain itu, sertifikat keamanan dapat dilihat langsung melalui browser pada bagian certificate website.



```
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ curl -k https://localhost  
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<meta name="viewport" content="width=device-width, initial-scale=1.0">  
<title>KELOMPOK 5</title>  
<link rel="preconnect" href="https://fonts.googleapis.com">  
<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600;700&display=swap" rel="stylesheet">  
<style>  
*{  
margin:0;  
padding:0;  
box-sizing:border-box;  
font-family:'Poppins',sans-serif;  
scroll-behavior:smooth;  
}  
body{  
background:linear-gradient(135deg,#0a0e27 0%,#0b1020 50%,#140b2d 100%);  
color:white;  
overflow-x:hidden;  
position:relative;  
}  
/* Animated Background */  
body::before{  
content:'';  
position:fixed;  
top:0;  
left:0;  
width:100%;  
height:100%;  
background:  
radial-gradient(circle at 20% 50%,rgba(0,247,255,0.1) 0%,transparent 50%),  
radial-gradient(circle at 80% 80%,rgba(147,112,219,0.1) 0%,transparent 50%);  
animation:bgShift 15s ease-in-out infinite;  
}
```

**Gamba 4.2.1** Code SSL/TLS Valid(HTTPS)



*Gambar 4.2.2 Website*

### 4.3 Fail2Ban Aktif

Fail2Ban berhasil diaktifkan untuk melindungi layanan SSH dari serangan brute force. Sistem akan memblokir alamat IP yang melakukan percobaan login gagal secara berulang sesuai konfigurasi yang telah ditentukan.

Verifikasi dilakukan menggunakan perintah:

```
sudo fail2ban-client status
```

Hasil pengujian menunjukkan bahwa layanan Fail2Ban aktif dan berjalan dengan baik pada server.

```
ubuntu@ubuntu: ~$ sudo systemctl status fail2ban
[sudo] password for ubuntu:
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-05-26 11:36:16 UTC; 22h ago
     Docs: man:fail2ban(1)
    Main PID: 114093 (fail2ban-server)
      Tasks: 5 (limit: 4600)
    Memory: 21.0M (peak: 25.9M)
       CPU: 1min 30.963s
    CGroup: /system.slice/fail2ban.service
            └─114093 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

May 26 11:36:16 ubuntu systemd[1]: Started fail2ban.service - Fail2Ban Service.
May 26 11:36:17 ubuntu fail2ban-server[114093]: 2026-05-26 11:36:17,003 fail2ban.configreader [114093]: WARNING 'allowip6' not defined in 'Definition'.
May 26 11:36:17 ubuntu fail2ban-server[114093]: Server ready

ubuntu@ubuntu:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
  '- Jail list:        sshd
```

**Gambar 4.3 Fail2Ban aktif**

#### 4.4 OS Hardening (sysctl)

Sistem operasi Linux telah menerapkan konfigurasi hardening menggunakan `sysctl` untuk meningkatkan keamanan jaringan dan sistem operasi. Konfigurasi ini bertujuan untuk mengurangi potensi serangan jaringan dan meningkatkan keamanan server secara keseluruhan.

Beberapa konfigurasi keamanan jaringan berhasil diterapkan dan diverifikasi melalui file konfigurasi sistem.



```
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ sudo nano /etc/sysctl.conf  
ubuntu@ubuntu:~$ sudo sysctl -p  
net.ipv4.icmp_echo_ignore_broadcasts = 1  
net.ipv4.conf.all.accept_redirects = 0  
net.ipv4.conf.all.send_redirects = 0  
net.ipv4.conf.all.accept_source_route = 0  
net.ipv4.tcp_syncookies = 1
```

**Gambar 4.4 OS hardening (sysctl)**

#### 4.5 Database Tidak Diekspos ke Luar Host

Sistem berhasil dikonfigurasi agar hanya port tertentu yang terbuka untuk publik. Container database MariaDB tidak membuka akses port ke luar host sehingga database hanya dapat diakses melalui internal Docker network.

Verifikasi dilakukan menggunakan perintah:

`ss -tulpn`

Hasil pengujian menunjukkan bahwa tidak terdapat port `0.0.0.0:3306` yang terbuka ke publik sehingga keamanan database lebih terjaga.



```

ubuntu@ubuntu:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
5a8b26ae3100   nginx:latest "/docker-entrypoint..." 31 minutes ago Up 19 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:
3293140c18dc   nginx-proxy "/docker-entrypoint.s..." 31 minutes ago Up 31 minutes 80/tcp
638e4b6d76b9   wordpress:latest "/docker-entrypoint.s..." 31 minutes ago Up 31 minutes 3306/tcp
638e4b6d76b9   mariadb:11   "/docker-entrypoint.s..." 31 minutes ago Up 31 minutes 3306/tcp
ubuntu@ubuntu:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
0840ba528888   bridge    bridge    local
f2bbdc2b9c5    host      host      local
d428077c9525   none      null      local
c553d524b078   ubuntu_backend bridge    local
3f3bf1723a98   ubuntu_frontend bridge    local
ubuntu@ubuntu:~$ sudo ufw status
Status: active

To               Action      From
--
22/tcp           ALLOW       Anywhere
80/tcp           ALLOW       Anywhere
443/tcp          ALLOW       Anywhere
53               ALLOW       Anywhere
22/tcp (v6)      ALLOW       Anywhere (v6)
80/tcp (v6)      ALLOW       Anywhere (v6)
443/tcp (v6)     ALLOW       Anywhere (v6)
53 (v6)          ALLOW       Anywhere (v6)

```

**Gambar 4.6** Output perintah: `docker ps`, `docker network ls`, `ufw status`

## 4.7 Network Bridge Isolation

Sistem menggunakan Docker bridge network yang dipisahkan menjadi frontend dan backend network. Pemisahan network ini bertujuan untuk meningkatkan keamanan komunikasi antar container dan membatasi akses layanan tertentu.

Container MariaDB hanya terhubung ke backend network sehingga tidak dapat diakses langsung dari luar server.

```

ubuntu@ubuntu:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
0840ba528888   bridge    bridge    local
f2bbdc2b9c5    host      host      local
d428077c9525   none      null      local
c553d524b078   ubuntu_backend bridge    local
3f3bf1723a98   ubuntu_frontend bridge    local

```

**Gambar 4.7** Network Bridge Isolation

## 4.8 Kontainer Berjalan Non-Root

Container dijalankan dengan memperhatikan aspek keamanan agar layanan tidak berjalan menggunakan akses root secara langsung. Hal ini bertujuan untuk meminimalkan risiko apabila terjadi eksploitasi pada container yang berjalan.

Verifikasi dilakukan dengan pengecekan user pada container WordPress dan menunjukkan bahwa container berjalan menggunakan user selain root.

```
ubuntu@ubuntu:~$ docker compose up -d
[*] up 3/3
✓Container mariadb      Running      0.0s
✓Container wordpress    Started      1.6s
✓Container nginx-proxy   Running      0.0s
ubuntu@ubuntu:~$ docker exec -it wordpress whoami
www-data
```

*Gambar 4.8 Kontainer berjalan Non-root*

## 4.9 File docker-compose.yml Valid

Container dijalankan dengan memperhatikan aspek keamanan agar layanan tidak berjalan menggunakan akses root secara langsung. Hal ini bertujuan untuk meminimalkan risiko apabila terjadi eksploitasi pada container yang berjalan.

Verifikasi dilakukan dengan pengecekan user pada container WordPress dan menunjukkan bahwa container berjalan menggunakan user selain root.

```
ubuntu@ubuntu:~$ docker compose config
name: ubuntu
services:
  mariadb:
    container_name: mariadb
    environment:
      MYSQL_DATABASE: wordpressdb
      MYSQL_PASSWORD: "12345"
      MYSQL_ROOT_PASSWORD: "12345"
      MYSQL_USER: wpuser
    expose:
      - "3306"
    image: mariadb:11
    networks:
      backend: null
    restart: always
    volumes:
      - type: volume
        source: mariadb_data
        target: /var/lib/mysql
        volume: {}
  nginx:
    container_name: nginx-proxy
    depends_on:
      wordpress:
        condition: service_started
        required: true
    image: nginx:latest
    networks:
      backend: null
      frontend: null
    ports:
      - mode: ingress
        target: 80
        published: "80"
        protocol: tcp
      - mode: ingress
        target: 443
        published: "443"
        protocol: tcp
    restart: always
```

*Gambar 4.9 docker-compose.yml valid*

## **BAB V**

### **KENDALA DAN SOLUSI**

#### **5.1 Kendala yang Dihadapi**

Selama proses pengerjaan tugas besar ini, terdapat beberapa kendala yang dihadapi oleh kelompok kami. Kendala pertama adalah kondisi jaringan internet di lingkungan kampus yang kurang stabil dan sering mengalami keterlambatan (lag), terutama saat melakukan proses instalasi package, pengunduhan image Docker, serta konfigurasi layanan berbasis container.

Kendala kedua adalah pengaturan waktu pengerjaan antar anggota kelompok. Perbedaan jadwal perkuliahan dan aktivitas masing-masing anggota menyebabkan koordinasi pengerjaan menjadi cukup menantang.

#### **5.2 Solusi yang Diterapkan**

Untuk mengatasi kendala jaringan internet, kelompok kami melakukan pengerjaan secara bertahap di luar jam sibuk penggunaan jaringan serta memanfaatkan koneksi internet alternatif agar proses konfigurasi dapat berjalan lebih lancar.

Sementara itu, untuk mengatasi kendala koordinasi antar anggota, kelompok kami membagi tugas sesuai tanggung jawab masing-masing anggota dan melakukan koordinasi secara berkala melalui diskusi daring maupun tatap muka sehingga seluruh proses pengerjaan tetap dapat diselesaikan dengan baik dan tepat waktu.

## BAB VI

### PEMBAGIAN TUGAS ANGGOTA

Dalam pelaksanaan tugas besar ini, setiap anggota kelompok memiliki peran dan tanggung jawab masing-masing agar proses pengerjaan dapat berjalan secara terstruktur dan efisien. Pembagian tugas dilakukan berdasarkan kemampuan dan kontribusi setiap anggota dalam pengembangan sistem maupun penyusunan laporan.

- **Dolly Efredi Bukit**  
Bertindak sebagai Project Manager sekaligus berkontribusi dalam proses konfigurasi dan implementasi sistem berbasis Docker serta layanan web server yang digunakan pada proyek.
- **Cindy Samosir**  
Bertanggung jawab dalam proses implementasi sistem, konfigurasi layanan container, pengujian layanan, serta penyusunan laporan teknis dan dokumentasi hasil pengerjaan.
- **Rohaya Hasibuan**  
Berkontribusi dalam penyusunan laporan teknis, dokumentasi konfigurasi, serta pengumpulan screenshot hasil pengujian sistem sesuai kebutuhan laporan.
- **Cindy Artika**  
Bertanggung jawab dalam penyusunan dan perapihan laporan akhir, termasuk pengaturan format laporan dan dokumentasi hasil implementasi sistem.
- **Chaterine Eklesia Maryati**  
Bertugas dalam pembuatan desain arsitektur sistem, diagram jaringan, serta ilustrasi hubungan antar container yang digunakan pada proyek.
- **Earlin Tasia Gulo**  
Berkontribusi dalam proses pengujian sistem, pengecekan hasil konfigurasi layanan, serta membantu proses validasi output layanan yang ditampilkan pada browser dan terminal.

## **BAB VII**

### **PENUTUP**

#### **7.1 Kesimpulan**

Berdasarkan hasil implementasi yang telah dilakukan, sistem web berbasis container menggunakan Docker berhasil dibangun dan dijalankan dengan baik menggunakan tiga container utama, yaitu Nginx sebagai reverse proxy, WordPress sebagai aplikasi web, dan MariaDB sebagai database server.

Selain itu, beberapa mekanisme keamanan juga berhasil diterapkan pada sistem, seperti penggunaan HTTPS dengan SSL/TLS, konfigurasi firewall UFW, SSH Hardening, serta Fail2Ban untuk meningkatkan keamanan server dan jaringan. Database juga berhasil diamankan dengan tidak mengekspos port database ke luar host serta menerapkan Docker bridge network untuk isolasi komunikasi antar container.

Dengan implementasi tersebut, sistem yang dibangun telah memenuhi kebutuhan tugas besar mata kuliah Keamanan Server dan Jaringan serta mampu berjalan dengan baik dan aman sesuai konfigurasi yang telah dirancang.

#### **7.2 Saran**

Pengembangan selanjutnya dapat dilakukan dengan menggunakan sertifikat SSL resmi seperti Let's Encrypt agar keamanan HTTPS lebih optimal. Selain itu, sistem juga dapat dikembangkan dengan menambahkan monitoring server, backup otomatis, serta mekanisme keamanan tambahan untuk meningkatkan kestabilan dan keamanan layanan.

## **LAMPIRAN**

### **Link Repository GitHub**

Repository proyek dapat diakses melalui link berikut : <https://github.com/Dolly-021/kel-5-ksj.git>

### **Link Video Presentasi**

Video presentasi tugas besar dapat diakses melalui link berikut: <https://youtu.be/b6Q-v14gGUQ>